

JS Functions

1. What is Function?

- a. A **function** in Javascript is a block of code written to perform a specific or particular task.
- b. It is executed when it is called or invoked.
- c. Functions help to avoid repetition of code and make it reusable.
- d. **Syntax:**



```
function functionName(parameters) {  
    // function body  
    // code to be executed  
}
```

2. Why do we need functions in JS?

- a. Functions are necessary in JavaScript to:
 - i. Make code reusable and modular.
 - ii. Organize code into manageable blocks.
 - iii. Avoid repetition.
 - iv. Improve readability and maintainability of the code.

3. What is Parameter?

- a. A **parameter** is a variable defined in a function's definition.
- b. It acts as a placeholder for the value that will be passed to the function.

c. For Example:

```
1 function greet(name) { // Here, 'name' is the parameter
2   console.log("Hello, " + name);
3 }
```

4. What is Argument?

a. An **argument** is the actual value that is passed to a function when it is called. Arguments replace the function's parameters.

b. For Example:

```
1 function greet(name) {
2   console.log("Hello, " + name);
3 }
4
5 greet("John"); // Here, 'John' is the argument
```

5. What is a Function Call Statement?

a. A **function call statement** is the part of the program where a function is invoked or called, meaning the function is executed.

b. To call a function, you simply use its function name followed by parentheses **()** which may include arguments (if required).

c. **General Syntax** →

 **functionName(arguments);**

```
// Function call statement
greet("Alice"); // Output: Hello, Alice
```

6. **this** Keyword in Function:

- a. **this** keyword in a normal function refers to the *Global Object* (*Window Object*).
- b. In non-strict mode:
 - i. In browsers, that's *window*.
 - ii. In Node.js, that's *global*.
- c. In strict mode ('**use strict**'):
 - i. **this** will be *undefined* when a normal function is called without an owner object.
- d. **For Example:**

```
function demo() {  
    console.log(this); // Window Object  
}  
  
demo();
```

7. Types of Functions in JS:

- a. *There are several types of functions in JavaScript:*
 - i. Named Functions (Function Declaration):
 - ii. Anonymous Function:
 - iii. Function Expressions:
 - iv. Arrow Functions:
 - v. IIFE (Immediately Invoked Function Expressions):
 - vi. Callback Function:
 - vii. Higher-order Functions:
 - viii. Nested Function:
 - ix. Generator Functions:

8. Named Functions (Function Declaration):

- a. A function that is defined with a name is called a Named Function.
- b. In other words we can say that a function which have identifier is known as Named Function.
- c. **Syntax** →



```
function identifier (params) {  
    // function body  
    // code to be executed  
}
```

d. **Parameterized Function:**

- i. A function that accepts parameters is called as Parameterized Function.
- ii. **For Example:**



```
1 function greet(name) {  
2   console.log("Hello, " + name);  
3 }  
4  
5 greet("Alice"); // Output: Hello, Alice
```

e. **Non-Parameterized Function:**

- i. A function that does not accept parameters is called as Non-parameterized Function.
- ii. **For Example:**



```
1 function greet() {  
2   console.log("Hello, World!");  
3 }  
4  
5 greet(); // Output: Hello, World!
```

f. **What is return Keyword in Function?:**

- i. The **return** keyword is used to send a value from a function back to the caller.
- ii. When a return statement is executed, the function stops executing.
- iii. **For Example:**

```
1 function add(a, b) {  
2   return a + b;  
3 }  
4  
5 console.log(add(5, 3)); // Output: 8
```

9. Anonymous Function:

- a. An anonymous function is a function that does not have a name.
- b. **For Example** →

```
1 (function () {  
2   console.log("This is Anonymous Function!");  
3 }());
```

- c. You can still call the anonymous function without storing inside the variable. You have to just wrap the function with the help of **() parenthesis** and call the function inside the parenthesis by using empty function call statements or parenthesis **()**.



```
1  const greet = function () {  
2    console.log("Hello, World!");  
3  };  
4  
5  greet(); // Output: Hello, World!
```

10. Function Expressions:

- a. A function which is passed or assigned as an expression to the variable is known as Function Expression.
- b. For Example →



```
1  const greet = function (name) {  
2    console.log("Hello, " + name);  
3  };  
4  
5  greet("John"); // Output: Hello, John
```

11. IIFE (Immediately Invoked Function Expressions):

- a. An IIFE is a function that is defined and immediately called.
- b. IIFE is a function which is called just after the its declaration.
- c. IIFE function will execute **only once**.
- d. For Example →



```
1  (function () {  
2    console.log("This is an IIFE!");  
3  })(); // Output: This is an IIFE!
```

12. Arrow Functions: (Fat Arrow Function)

- a. Arrow functions are a shorter way of writing functions.
- b. It is especially useful for anonymous functions.
- c. They are anonymous and do not have their own *this*.
- d. It is introduced in the *ES6* version which was released in the year *2015*.
- e. Arrow functions do not have an *arguments* object.
- f. **Syntax** →



```
const functionName = (parameters) ⇒ {  
    // function body  
    // code to be executed  
}
```

- g. **For Example** →



```
1  const greet = (name) ⇒ {  
2    console.log("Hello, " + name);  
3  };  
4  
5  greet("Bob"); // Output: Hello, Bob
```

- h. **Implicit return on a single execution statement:**

- i. If the function has a single statement, you can omit the *return* and *curly braces {}*.
- ii. **For Example** →



```
1  const add = (a, b) ⇒ a + b;  
2  
3  console.log(add(3, 4)); // Output: 7
```

i. **Explicit return:**

- i. If there are multiple statements, you need to use **return** explicitly and you have to use **curly braces {}** mandatory.

ii. **For Example** →

```
1  const add = (a, b) => {  
2    return a + b;  
3  };  
4  
5  console.log(add(3, 4)); // Output: 7
```

13. What is arguments Object?

- a. In traditional functions (not arrow functions), JavaScript provides an **arguments** object which is an array-like object representing the passed arguments.

b. **For Example** →

```
1  function sum(a,b,c,d,e) {  
2    console.log(arguments);  
3    console.log(Array.isArray(arguments)); // false  
4  }  
5  sum(1,2,3,4,5);  
6  // Output: Arguments(5) [1, 2, 3, 4, 5, callee: f, Symbol(Symbol.iterator): f]
```

- c. It will return an array-like object but it is not an actual array.
- d. **NOTE:** There is no arguments object for arrow function for that you have to use **rest parameter**.

14. Callback Function:

- a. A *callback function* is a function passed into another function as an *argument*, and it gets executed after the parent function finishes.
- b. For Example →



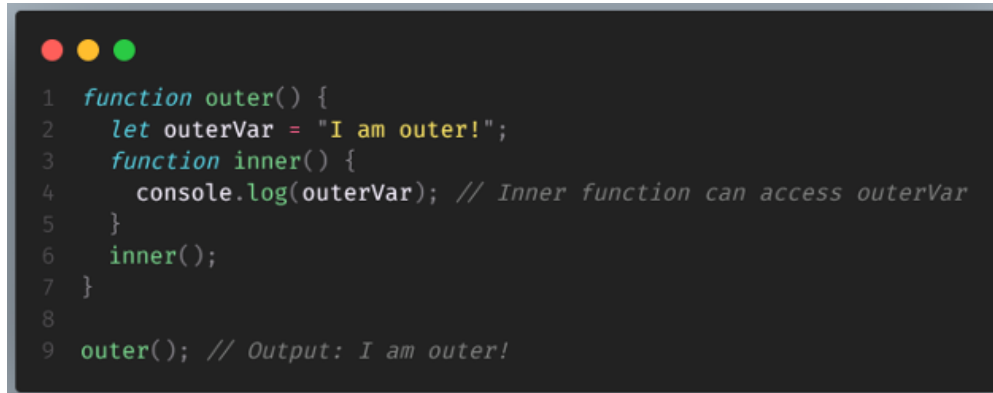
```
1  function greet(name, callback) {  
2    console.log("Hello, " + name);  
3    callback();  
4  }  
5  
6  function callFun() {  
7    console.log("How are you?");  
8  };  
9  
10 greet("Alice", callFun);  
11 // Output:  
12 // Hello, Alice  
13 // How are you?
```

15. Higher-order Functions:

- a. A *higher-order function* is a function that either takes one or more functions as arguments or returns a function.
- b. *Here's a breakdown of what we can do with higher-order functions:*
 - i. It will take functions as arguments
 - ii. Return function from other functions

16. Nested Function:

- a. A function which is present inside another function is called as Nested Function.
- b. It can access variables from the outer function.
- c. That is something called as *closure* function.
- d. For Example →



```
1 function outer() {  
2   let outerVar = "I am outer!";  
3   function inner() {  
4     console.log(outerVar); // Inner function can access outerVar  
5   }  
6   inner();  
7 }  
8  
9 outer(); // Output: I am outer!
```

17. Generator Functions:

- a. A generator function is a special type of function that can be paused and resumed.
- b. It uses the *yield* keyword to pause the execution and hold the data.
- c. We can pause the execution of function with the help of generator function and whenever we want we can resume the same function.
- d. For creating generator function we have to use *** suffix as a function keyword.
- e. For Example → *function**
- f. For getting the value which is stored inside the yield we have to use *.next()* method.
- g. The *.next()* method will return one object with keys *value* and *done* as *true/false*.

h. For Example →

```
1 function* numbers() {
2   yield 1;
3   yield 2;
4   yield 3;
5   return 4;
6 }
7
8 const gen = numbers();
9 console.log(gen.next().value); // Output: 1
10 console.log(gen.next().value); // Output: 2
11 console.log(gen.next().value); // Output: 3
12 console.log(gen.next().value); // Output: 4
13 console.log(gen.next().value); // Output: undefined (because there is no next value present)
14
15 console.log(gen.next()); // Output: {value: undefined, done: true}
```

18. What is Closure Function?

- a. A **closure** in JavaScript is a feature where a function "remembers" its lexical environment (the variables from the scope where it was created) even after it is executed outside of that scope.
- b. Simply put, closures allow inner functions to access variables from their outer function, even after the outer function has finished execution.
- c. **NOTE:** *A closure is a function that has access to its own scope, the scope of the outer function, and the global scope.*
- d. **When will the Closure Function be Created?**
 - i. A **closure** itself is **not an object**; it is a **function**. Specifically, a closure occurs when a function is created inside another function and retains access to the variables from the outer function's scope, even after the outer function has finished execution.
 - ii. The closure is created when the **innerFunction** is returned from **outerFunction** and retains access to its lexical scope.

iii. For Example →

```
1 function outerFunction(outerVar) {
2   // This is the outer function
3
4   return function innerFunction(innerVar) {
5     // This is the inner function, which forms a closure
6     console.log("Outer Variable:", outerVar); // Can access outerVar
7     console.log("Inner Variable:", innerVar); // Can access innerVar
8   };
9 }
10
11 const closure = outerFunction("Hello"); // 'Hello' is passed to outerVar
12 closure("World"); // Output: Outer Variable: Hello, Inner Variable: World
13
14 // OR
15 // Directly calling innerFunction inside outerFunction
16 outerFunction("Hello")("World");
```

19. Function Currying:

- a. **Function Currying** is transforming a function that takes multiple arguments into a sequence of nested functions, each taking *one argument at a time*.

b. OR

- c. Currying is breaking a function that takes multiple arguments into a series of functions that each take one argument, returning another function until all arguments are provided.

d. For Example → Normal Function

```
function add(a, b, c) {
  return a + b + c;
}
console.log(add(2, 3, 4)); // 9
```

e. Curried Version:

```
function add(a) {  
  return function(b) {  
    return function(c) {  
      return a + b + c;  
    }  
  }  
}
```

```
console.log(add(2)(3)(4)); // 9
```

f. Here,

- i. First call `add(2)` → returns a function waiting for `b`.
- ii. Then `(3)` → returns a function waiting for `c`.
- iii. Finally `(4)` → evaluates `2 + 3 + 4`.

g. Why use Currying?

- i. Reusability
- ii. Code clarity
- iii. Functional programming